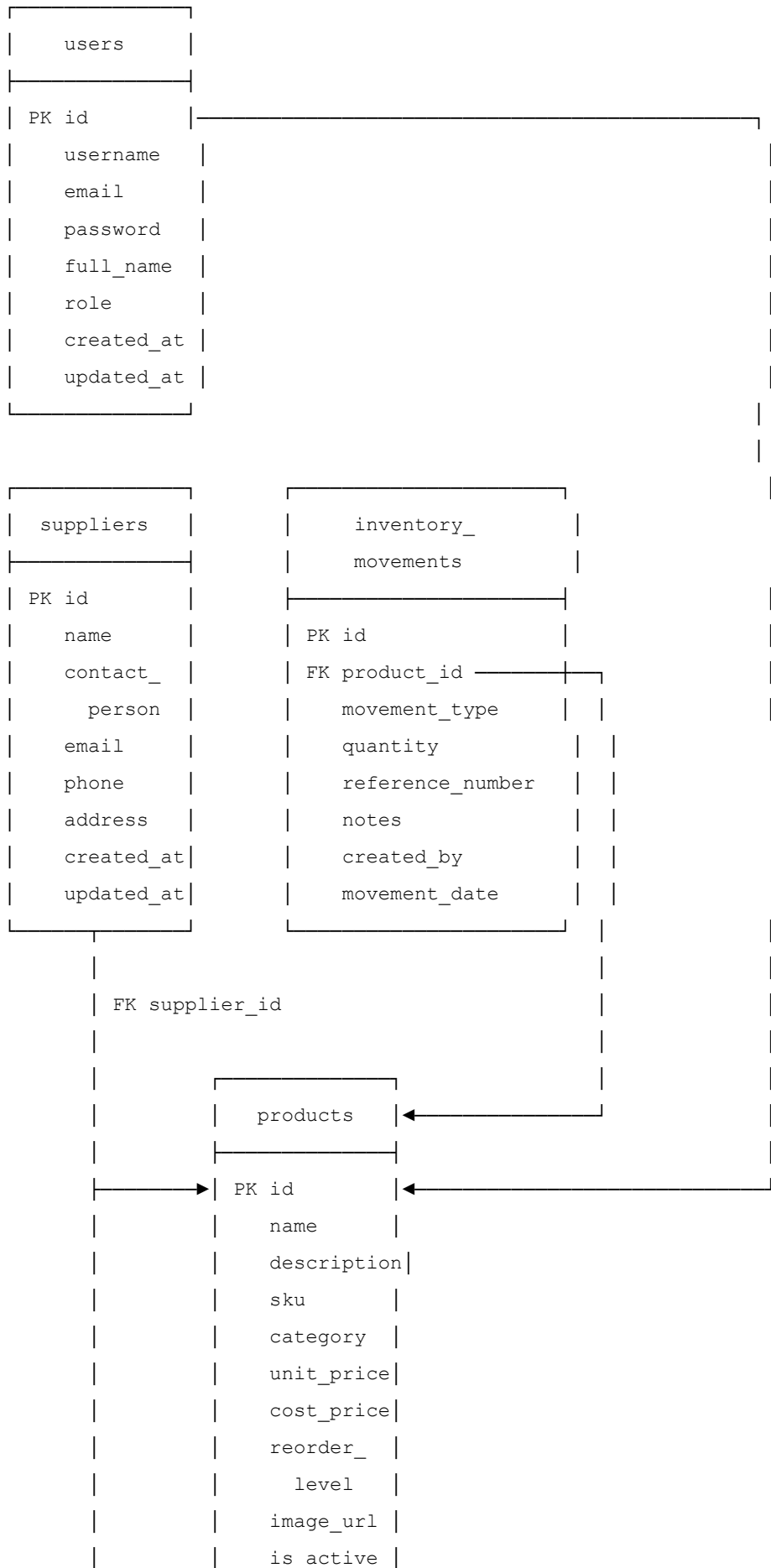


AI Inventory Agent - Database Design

Overview

- **DBMS:** MySQL 8.0+
 - **Database Name:** `ai_inventory`
 - **Character Set:** UTF-8 (default)
 - **Engine:** InnoDB (default, supports foreign keys and transactions)
 - **Connection:** `jdbc:mysql://localhost:3306/ai_inventory`
-

Entity-Relationship Diagram (Text-Based)



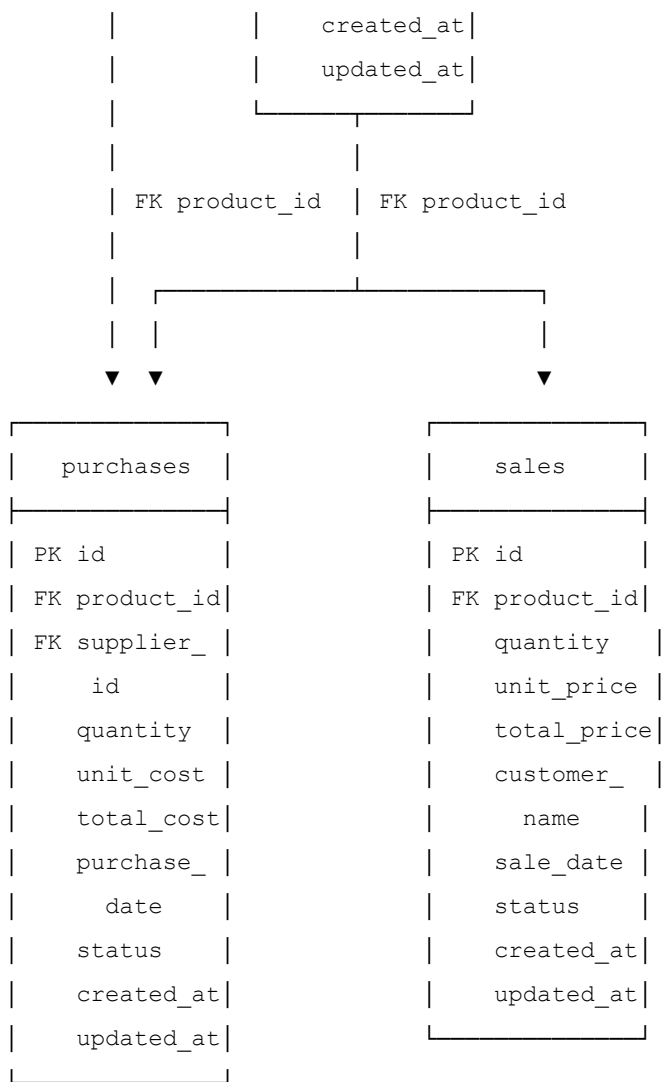


Table Descriptions

1. users

Stores registered users of the system.

Column	Type	Nullable	Default	Constraints
id	BIGINT	No	AUTO_INCREMENT	PRIMARY KEY
username	VARCHAR(50)	No	-	UNIQUE, NOT NULL
email	VARCHAR(100)	No	-	UNIQUE, NOT NULL
password	VARCHAR(255)	No	-	NOT NULL (BCrypt hashed)
full_name	VARCHAR(100)	Yes	NULL	
role	VARCHAR(20)	No	'USER'	NOT NULL
created_at	DATETIME	No	CURRENT_TIMESTAMP	NOT NULL, NOT UPDATABLE

Column	Type	Nullable	Default	Constraints
updated_at	DATETIME	No	CURRENT_TIMESTAMP	NOT NULL, auto-updates on change

Indexes:

- PRIMARY KEY (id)
- UNIQUE INDEX idx_users_username (username)
- UNIQUE INDEX idx_users_email (email)

JPA Entity: `com.inventory.agent.entity.User`

Validation Rules:

- username: 3-50 characters, required
- email: valid email format, required
- password: minimum 6 characters, required
- fullName: max 100 characters

2. products

Catalog of all inventory items.

Column	Type	Nullable	Default	Constraints
id	BIGINT	No	AUTO_INCREMENT	PRIMARY KEY
name	VARCHAR(100)	No	-	NOT NULL
description	TEXT	Yes	NULL	
sku	VARCHAR(50)	No	-	UNIQUE, NOT NULL
category	VARCHAR(50)	Yes	NULL	
unit_price	DECIMAL(10,2)	No	-	NOT NULL, POSITIVE
cost_price	DECIMAL(10,2)	Yes	NULL	POSITIVE
reorder_level	INT	No	0	NOT NULL
image_url	VARCHAR(255)	Yes	NULL	
is_active	BOOLEAN	No	TRUE	NOT NULL
created_at	DATETIME	No	CURRENT_TIMESTAMP	NOT NULL, NOT UPDATABLE
updated_at	DATETIME	No	CURRENT_TIMESTAMP	NOT NULL, auto-updates on change

Indexes:

- PRIMARY KEY (id)
- UNIQUE INDEX idx_products_sku (sku)

JPA Entity: `com.inventory.agent.entity.Product`

Validation Rules:

- name: max 100 characters, required
- sku: unique, required
- category: max 50 characters
- unitPrice: must be positive, required
- costPrice: must be positive
- reorderLevel: minimum 0
- description: max 500 characters

Notes:

- is_active is used for soft deletes. When a product is "deleted," this flag is set to FALSE rather than removing the row. This preserves referential integrity for historical sales and purchase records.

3. suppliers

External vendors that supply products.

Column	Type	Nullable	Default	Constraints
id	BIGINT	No	AUTO_INCREMENT	PRIMARY KEY
name	VARCHAR(100)	No	-	NOT NULL
contact_person	VARCHAR(100)	Yes	NULL	
email	VARCHAR(100)	Yes	NULL	
phone	VARCHAR(20)	Yes	NULL	
address	TEXT	Yes	NULL	
created_at	DATETIME	No	CURRENT_TIMESTAMP	NOT NULL, NOT UPDATABLE
updated_at	DATETIME	No	CURRENT_TIMESTAMP	NOT NULL, auto-updates

Indexes:

- PRIMARY KEY (id)

JPA Entity: com.inventory.agent.entity.Supplier

Validation Rules:

- name: max 100 characters, required
- contactPerson: max 100 characters
- email: valid email format
- phone: max 20 characters
- address: max 255 characters

4. inventory_movements

Audit log of all stock changes (IN and OUT).

Column	Type	Nullable	Default	Constraints
id	BIGINT	No	AUTO_INCREMENT	PRIMARY KEY
product_id	BIGINT	No	-	FOREIGN KEY -> products(id)
movement_type	VARCHAR(20)	No	-	NOT NULL ('IN' or 'OUT')
quantity	INT	No	-	NOT NULL, POSITIVE
reference_number	VARCHAR(50)	Yes	NULL	
notes	TEXT	Yes	NULL	
created_by	VARCHAR(50)	Yes	NULL	
movement_date	DATETIME	No	CURRENT_TIMESTAMP	NOT NULL

Indexes:

- PRIMARY KEY (id)
- INDEX idx_movements_product_id (product_id)
- INDEX idx_movements_type (movement_type)
- INDEX idx_movements_date (movement_date)

JPA Entity: `com.inventory.agent.entity.InventoryMovement`

Custom Repository Queries:

```
// Total IN quantity for a product
@Query("SELECT COALESCE(SUM(im.quantity), 0) FROM InventoryMovement im
        WHERE im.product.id = :productId AND im.movementType = 'IN'")
Integer findTotalInByProductId(@Param("productId") Long productId);

// Total OUT quantity for a product
@Query("SELECT COALESCE(SUM(im.quantity), 0) FROM InventoryMovement im
        WHERE im.product.id = :productId AND im.movementType = 'OUT'")
Integer findTotalOutByProductId(@Param("productId") Long productId);
```

Notes:

- Current stock for a product = SUM(IN movements) - SUM(OUT movements).
- Movement types are 'IN' (stock received) and 'OUT' (stock dispatched).
- `reference_number` links movements to specific business events (e.g., "SALE-1", "PO-001").

5. sales

Records of products sold to customers.

Column	Type	Nullable	Default	Constraints
id	BIGINT	No	AUTO_INCREMENT	PRIMARY KEY
product_id	BIGINT	No	-	FOREIGN KEY -> products(id)
quantity	INT	No	-	NOT NULL, POSITIVE
unit_price	DECIMAL(10,2)	No	-	NOT NULL, POSITIVE
total_price	DECIMAL(10,2)	No	-	NOT NULL (auto-calculated)
customer_name	VARCHAR(100)	Yes	NULL	
sale_date	DATETIME	No	CURRENT_TIMESTAMP	NOT NULL
status	VARCHAR(20)	No	'PENDING'	NOT NULL
created_at	DATETIME	No	CURRENT_TIMESTAMP	NOT NULL, NOT UPDATABLE
updated_at	DATETIME	No	CURRENT_TIMESTAMP	NOT NULL, auto-updates

Indexes:

- PRIMARY KEY (id)
- INDEX idx_sales_product_id (product_id)
- INDEX idx_sales_date (sale_date)
- INDEX idx_sales_status (status)

JPA Entity: com.inventory.agent.entity.Sale

Custom Repository Queries:

```
// Sales within a date range
List<Sale> findBySaleDateBetween(LocalDateTime start, LocalDateTime end);

// Daily sales totals within a date range
@Query("SELECT s.saleDate, SUM(s.totalAmount) FROM Sale s
        WHERE s.saleDate BETWEEN :start AND :end
        GROUP BY s.saleDate ORDER BY s.saleDate")
List<Object[]> findTotalSalesByDateRange(
    @Param("start") LocalDateTime start,
    @Param("end") LocalDateTime end);

// Top selling products by quantity
@Query("SELECT s.product, SUM(s.quantity) as totalQty FROM Sale s
        GROUP BY s.product ORDER BY totalQty DESC")
List<Object[]> findTopSellingProducts();
```

Business Logic:

- `totalPrice` is automatically computed via `@PrePersist` and `@PreUpdate` lifecycle callbacks: `unitPrice * quantity`.
- Creating a sale via the service layer also generates an inventory OUT movement with reference `"SALE-{id}"`.
- The service validates sufficient stock before allowing a sale.

6. purchases

Records of products ordered from suppliers.

Column	Type	Nullable	Default	Constraints
<code>id</code>	<code>BIGINT</code>	No	<code>AUTO_INCREMENT</code>	PRIMARY KEY
<code>product_id</code>	<code>BIGINT</code>	No	-	FOREIGN KEY -> <code>products(id)</code>
<code>supplier_id</code>	<code>BIGINT</code>	No	-	FOREIGN KEY -> <code>suppliers(id)</code>
<code>quantity</code>	<code>INT</code>	No	-	NOT NULL, POSITIVE
<code>unit_cost</code>	<code>DECIMAL(10,2)</code>	No	-	NOT NULL, POSITIVE
<code>total_cost</code>	<code>DECIMAL(10,2)</code>	No	-	NOT NULL (auto-calculated)
<code>purchase_date</code>	<code>DATETIME</code>	No	<code>CURRENT_TIMESTAMP</code>	NOT NULL
<code>status</code>	<code>VARCHAR(20)</code>	No	<code>'ORDERED'</code>	NOT NULL
<code>created_at</code>	<code>DATETIME</code>	No	<code>CURRENT_TIMESTAMP</code>	NOT NULL, NOT UPDATABLE
<code>updated_at</code>	<code>DATETIME</code>	No	<code>CURRENT_TIMESTAMP</code>	NOT NULL, auto-updates

Indexes:

- PRIMARY KEY (`id`)
- INDEX `idx_purchases_product_id` (`product_id`)
- INDEX `idx_purchases_supplier_id` (`supplier_id`)
- INDEX `idx_purchases_status` (`status`)
- INDEX `idx_purchases_date` (`purchase_date`)

JPA Entity: `com.inventory.agent.entity.Purchase`

Custom Repository Queries:

```
List<Purchase> findByProductId(Long productId);
List<Purchase> findBySupplierId(Long supplierId);
List<Purchase> findByStatus(String status);
List<Purchase> findByPurchaseDateBetween(LocalDateTime start, LocalDateTime end);
```


Business Logic:

- `totalCost` is automatically computed via `@PrePersist` and `@PreUpdate: unitCost * quantity`.
- **Status flow:** ORDERED -> SHIPPED -> RECEIVED (or CANCELLED at any point).

Relationships

Foreign Key Relationships

```
products.id <— inventory_movements.product_id    (One-to-Many)
products.id <— sales.product_id                    (One-to-Many)
products.id <— purchases.product_id                (One-to-Many)
suppliers.id <— purchases.supplier_id              (One-to-Many)
```

Relationship Descriptions

Parent Table	Child Table	Relationship Cardinality	On Delete Behavior
products	inventory_movements	One-to-Many 0..N	Restrict/Cascade
products	sales	One-to-Many 0..N	Restrict/Cascade
products	purchases	One-to-Many 0..N	Restrict/Cascade
suppliers	purchases	One-to-Many 0..N	Restrict/Cascade

JPA Fetch Strategy

All `@ManyToOne` relationships use `FetchType.LAZY` to prevent N+1 query issues:

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "product_id", nullable = false)
private Product product;
```

Indexes Summary

Table	Index Name	Columns	Type
users	PRIMARY	id	Primary
users	idx_users_username	username	Unique
users	idx_users_email	email	Unique
products	PRIMARY	id	Primary
products	idx_products_sku	sku	Unique
suppliers	PRIMARY	id	Primary
inventory_movements	PRIMARY	id	Primary

Table	Index Name	Columns	Type
inventory_movements	idx_movements_product_id	product_id	Non-unique
inventory_movements	idx_movements_type	movement_type	Non-unique
inventory_movements	idx_movements_date	movement_date	Non-unique
sales	PRIMARY	id	Primary
sales	idx_sales_product_id	product_id	Non-unique
sales	idx_sales_date	sale_date	Non-unique
sales	idx_sales_status	status	Non-unique
purchases	PRIMARY	id	Primary
purchases	idx_purchases_product_id	product_id	Non-unique
purchases	idx_purchases_supplier_id	supplier_id	Non-unique
purchases	idx_purchases_status	status	Non-unique
purchases	idx_purchases_date	purchase_date	Non-unique

Schema Management

The project uses Hibernate's `ddl-auto=update` strategy, meaning:

- Hibernate automatically creates tables on first run.
- On subsequent runs, Hibernate **adds** new columns/tables but does **not** remove or modify existing ones.
- For production, consider using a migration tool like **Flyway** or **Liquibase** for controlled schema evolution.

```
spring.jpa.hibernate.ddl-auto=update
```

Stock Calculation Logic

Stock levels are **not stored** in a dedicated column. Instead, they are computed at runtime:

```
Current Stock = SUM(quantity WHERE movement_type = 'IN')
               - SUM(quantity WHERE movement_type = 'OUT')
```

This is implemented via two repository queries:

```
@Query("SELECT COALESCE(SUM(im.quantity), 0) FROM InventoryMovement im
        WHERE im.product.id = :productId AND im.movementType = 'IN'")
Integer findTotalInByProductId(@Param("productId") Long productId);

@Query("SELECT COALESCE(SUM(im.quantity), 0) FROM InventoryMovement im
        WHERE im.product.id = :productId AND im.movementType = 'OUT'")
Integer findTotalOutByProductId(@Param("productId") Long productId);
```

Connection Pool Configuration

The application uses **HikariCP** as the connection pool (default with Spring Boot):

```
spring.datasource.hikari.maximum-pool-size=20
spring.datasource.hikari.minimum-idle=5
spring.datasource.hikari.idle-timeout=300000      # 5 minutes
spring.datasource.hikari.max-lifetime=1200000    # 20 minutes
```

Data Type Mapping (Java to MySQL)

Java Type	MySQL Type	JPA Column Annotation
Long	BIGINT	@Id @GeneratedValue(IDENTITY)
String	VARCHAR(n)	@Column(length = n)
String (large)	TEXT	@Column(length = 500)
Integer	INT	@Column
BigDecimal	DECIMAL(10,2)	@Column(precision=10, scale=2)
Boolean	BOOLEAN/TINYINT	@Column
LocalDateTime	DATETIME	@PrePersist / @PreUpdate